

WindBorne Firmware Challenge — Joshua Oliveira

1. A cool thing I built

NVDA	Market	sell	1.00	1.00	177.11	filled	access_key	Apr 02, 2026, 12:18:25 PM	Apr 02, 2026, 12:18:26 PM
NVDA	Market	buy	1.00	1.00	177.27	filled	access_key	Apr 02, 2026, 12:16:08 PM	Apr 02, 2026, 12:16:08 PM
NVDA	Market	sell	1.00	1.00	177.23	filled	access_key	Apr 02, 2026, 12:13:52 PM	Apr 02, 2026, 12:13:52 PM
NVDA	Market	buy	1.00	1.00	177.45	filled	access_key	Apr 02, 2026, 12:11:22 PM	Apr 02, 2026, 12:11:22 PM
NVDA	Market	sell	1.00	1.00	177.09	filled	access_key	Apr 02, 2026, 12:09:05 PM	Apr 02, 2026, 12:09:05 PM
NVDA	Market	buy	1.00	1.00	176.95	filled	access_key	Apr 02, 2026, 12:06:29 PM	Apr 02, 2026, 12:06:30 PM
NVDA	Market	sell	1.00	1.00	176.91	filled	access_key	Apr 02, 2026, 12:06:06 PM	Apr 02, 2026, 12:06:07 PM
NVDA	Market	buy	1.00	1.00	177.01	filled	access_key	Apr 02, 2026, 12:05:49 PM	Apr 02, 2026, 12:05:49 PM
NVDA	Market	sell	1.00	1.00	176.97	filled	access_key	Apr 02, 2026, 12:05:19 PM	Apr 02, 2026, 12:05:19 PM
NVDA	Market	buy	1.00	1.00	176.95	filled	access_key	Apr 02, 2026, 12:04:16 PM	Apr 02, 2026, 12:04:17 PM
NVDA	Market	sell	1.00	1.00	176.92	filled	access_key	Apr 02, 2026, 12:04:14 PM	Apr 02, 2026, 12:04:14 PM
NVDA	Market	buy	1.00	1.00	177.00	filled	access_key	Apr 02, 2026, 12:03:43 PM	Apr 02, 2026, 12:03:44 PM
NVDA	Market	sell	1.00	1.00	176.95	filled	access_key	Apr 02, 2026, 12:03:32 PM	Apr 02, 2026, 12:03:32 PM
NVDA	Market	buy	1.00	1.00	177.05	filled	access_key	Apr 02, 2026, 12:02:52 PM	Apr 02, 2026, 12:02:52 PM
NVDA	Market	sell	1.00	1.00	177.00	filled	access_key	Apr 02, 2026, 12:02:43 PM	Apr 02, 2026, 12:02:44 PM
NVDA	Market	buy	1.00	1.00	176.91	filled	access_key	Apr 02, 2026, 12:01:41 PM	Apr 02, 2026, 12:01:41 PM

I built an automated stock trading system because I was interested in trading but didn't know much about markets, and I wanted a project that would push me to learn more software development than I was getting from my coursework. Over time it evolved into a remotely deployed system that processes live market data, makes trading decisions autonomously, and runs on a server without my computer needing to be on. I'm especially proud of it because I taught myself a lot of the technologies involved, spent countless hours debugging obscure failures and deployment issues, and eventually got it operating on its own, which was the original goal from the start.

2. My setup

I mostly use VS Code because it works well across the different projects I tend to work on, whether that's Python, C++, or embedded systems work. I've also used Arduino IDE for Arduino development, Vivado for VHDL projects, and Git/GitHub extensively. I never really became attached to a particular editor; I picked VS Code because people said it was good, it looked nice, and it worked.

Over time, my workflow has changed much more than my editor choice. Early on I would make changes and push them quickly, but I gradually shifted toward local testing before deployment because it made debugging much easier and reduced the risk of breaking a working system. For my trading bot, I typically develop in VS Code, test locally, and use Git Bash to manage changes before deployment.

Most of my debugging happens through logs and instrumentation rather than the editor itself. My trading bot uses CSV logs, email alerts, and text alerts to make failures visible. On hardware projects I've relied on serial output, oscilloscopes, waveform generators, multimeters, and CSV analysis to understand system behavior. I've found that once a project becomes sufficiently complicated, understanding what the system is actually doing becomes much more important than the specific editor being used.

3. Debugging a PCB that randomly resets every few hours

My first instinct is usually to start with the simplest explanations before diving into more complicated failure modes. I would begin with a basic visual inspection of the board, looking for anything suspicious around power, reset circuitry, connectors, or areas where traces and components are unusually close together. My general debugging philosophy is to make the system observable before trying to fix it. For software, that usually means adding prints, logs, or instrumentation to narrow down where behavior starts diverging from expectations.

A similar issue actually came up during my autonomous vehicle capstone project. We designed custom PCBs where the traces were generated directly from the schematic, but the rivets connecting the layers had to be installed by hand. Looking at the board, some of the rivets seemed extremely close together, and after checking the rivet dimensions against our trace spacing I realized there was very little margin. To isolate the problem, we bypassed individual PCB traces with wires and tested the system one connection at a time. The software ran correctly when we bypassed the PCB, which told us the issue was hardware-related. After testing roughly seven traces we identified the problematic connection and confirmed that neighboring rivets were shorting together.

We also ran into a separate intermittent issue where wires plugged into the rivets would occasionally lose contact for a fraction of a second and reset the entire system. Experiences like that have made me a big believer in isolating variables and building simple experiments. If I couldn't find anything obvious through inspection, my next step would be to collect more information through logging, measurements, and targeted tests to determine whether the resets were caused by power, hardware, firmware, or something environmental. Once I have evidence, I usually change strategies based on what the system is telling me rather than following a fixed debugging checklist.

4. What firmware/embedded engineers sometimes do wrong

One thing I think engineers, especially students, do wrong is changing too many variables at once while debugging. If you modify ten things and the problem disappears, you don't actually know what fixed it. If it gets worse, you don't know what caused that either. I've found debugging is usually much faster when you isolate variables, make one change at a time, and treat each change like an experiment.

I also think engineers sometimes rush into implementation before thinking through how they are going to debug the system later. A thoughtful architecture, good observability, and clear debugging paths can save far more time than they cost. In my experience, many bugs end up having surprisingly simple root causes, but finding those root causes becomes much easier when the system was designed from the beginning to be understandable, testable, and observable.

A mistake I see fairly often is stopping once something works at a basic level. I would rather spend extra effort making a system easier to debug, validate, and maintain than save a few hours up front and spend days chasing problems later.

5. A sketchy fix that kept a project alive

One sketchy fix I used was in my automated trading bot when I was dealing with multithreading issues during restarts. Sometimes when the bot restarted, the old thread had not fully shut down before the new one started, which caused problems with duplicate startup behavior and connection handling. My temporary fix was to delay the new thread startup for about a minute so the previous thread had time to shut down cleanly.

I knew this was not the real architecture I wanted long term. A time delay is not the same thing as proper lifecycle management, thread ownership, or shutdown coordination. But it kept the system alive long enough for me to continue testing, understand the failure better, and eventually think more carefully about how the bot should manage background threads and long-running connections.

The thing I'm secretly more proud of, though, was probably my logging system. At one point I was struggling to understand why the bot was making technically valid but less-than-optimal trades. Instead of immediately rewriting the strategy logic, I started logging an absurd amount of information to CSV files: indicator values, strategy signals, confidence scores, trade decisions, and pretty much anything else I thought might be useful. It was definitely overkill, but it effectively turned the bot into a flight recorder. Once I had enough data, I could finally see why decisions were being made and where my assumptions were wrong. The logging infrastructure is still there today because it ended up being one of the most useful debugging tools in the entire project.

6. Why WindBorne?

What attracted me to WindBorne is that many of the engineering challenges sound similar to the types of projects I enjoy most: autonomous systems, limited observability, difficult-to-reproduce failures, and hardware/software integration. I enjoy building systems that operate independently and then figuring out why they behave the way they do when reality does not match expectations. The combination of firmware, experimentation, debugging, and real-world system behavior is what initially drew me toward computer engineering and why this role stood out to me.